

Proyecto Intermodular I

Proyecto Final

[Grupo]

Gestor de Eventos

Enunciado

Trabajando en grupo y con GIT - Github, siguiendo una metodología SCRUM, vamos a desarrollar, de manera incremental, fase a fase, un Gestor de Eventos siguiendo la siguiente arquitectura:

- Cada equipo elegirá un tipo de evento, no repetible. Es decir, dos equipos no podrán realizar el mismo Gestor de Eventos. Ejemplos:
 - Conciertos (música clásica, pop, rock...)
 - Deportes (fútbol, basket...)
 - Teatro
 - Conferencia
 - Arte
 - Cine
 - ... cualquier otro tipo de evento, a elegir por el grupo (no repetible), donde se tengan que vender entradas...
 - Modelos de datos a crear libremente con, al menos 8 - 10 campos (columnas) por cada modelo:
 - Usuarios (clientes que reservarán entradas)
 - Reservas (relacionará al cliente con el precio de la entrada)
 - Tarifas (tendrán un precio)
 - Los equipos serán de 2 ó 3 miembros. Según el número de integrantes:
 - Equipos de 3 miembros → Deberán realizar los 3 modelos de datos
 - Equipos de 2 miembros → Deberán realizar los 2 primeros modelos de datos (Usuarios y Reservas). Las Tarifas serán **opcionales**.
 - Alumnado con pérdida de evaluación continua → Lo mismo que el equipo de 2 miembros.
 - CRUDs obligatorios:
 - Usuarios → CRUD (Dar de alta, listar usuarios y sus detalles, actualizar datos de un usuario en particular y eliminarlos*)
 - Reservas → CRD (Crear una reserva, listar reservas, pero no sus detalles y eliminarlas, sin poder actualizarlas)
 - Tarifas*** → R (Sólo opción de listar** tarifas ya existentes en BBDD, previamente insertadas "a mano" con un script entregable)
- * Eliminar un usuario debería eliminar sus reservas (eliminado en cascada)*
*** Cualquier acción adicional que se desee realizar, es **valorable**.*
**** Si no hay Tarifas (equipos de 2 miembros) se añadirá, en Reservas, un campo "double" con el precio de la reserva puesto a mano.*
- Ejemplos de datos, para un Gestor de Cine, con los 3 modelos:
 - Un usuario compra una entrada con tarifa reducida
 - Se genera una reserva que relaciona ese usuario con esa tarifa reducida
 - A la hora de listar las reservas, podemos visualizar datos del usuario así como la tarifa elegida y su precio.

FASE 1 — BACKEND: API REST (SPRINGBOOT) + MYSQL DB

- API REST MVC SpringBoot – Hibernate JPA
- Modelos de datos:
 - Usuarios
 - Reservas
 - Tarifas * (según miembros de equipo)
- Configuración CORS
- Prueba de CRUDs con cliente REST (Postman, EchoAPI, Insomnia...)
- Crear documento PR, con TOKEN (git clone del BackEnd) y capturas de pantalla del cliente REST, demostrando que funciona.

FASE 2 — FRONTEND: (HTML + CSS + JS y/o REACT)

- Dashboard o landing page con 3 enlaces:
 - Usuarios → Listado de usuarios
 - Reservas → Listado de Reservas
 - Tarifas → Listado de Tarifas
- Cada listado tendrá las acciones de VER DETALLES (Show), EDITAR, ELIMINAR, según los CRUDs realizados en el API.
- Actualizar documento PR, con TOKEN (git clone del FrontEnd) y capturas de pantalla del FrontEnd, donde se demuestre que todos los circuitos del usuario (UX/UI) funcionan correctamente.

MÁS EXTRAS OPCIONALES A VALORAR

- Uso de **REACT** en FrontEnd
- Validación de datos (contraseñas, etc.)
- Estilos CSS interesantes (Bootstrap, Tailwind...)
- Mensajes visuales (SweetAlert2 y similares)
- Mensajes de confirmación al eliminar (¿Está seguro que desea eliminar...?)
- Cualquier funcionalidad extra que se os ocurra relacionada con un Gestor de Eventos.
- Código limpio
- Comentarios

- **Requisitos a tener en cuenta:**

- Si usáis IA, hacedlo como apoyo. El uso completo de la misma para desarrollar supondrá **una calificación de cero puntos para todo el grupo.**

- **Entrega:**

- Trabajad en equipo con GIT y GITHUB según la guía vista en clase:
 - Cread dos repositorios privados con un token asociado (Uno para backend y otro para frontend)
 - Trabajad en Git Local cada miembro del equipo creando vuestras propias ramas de desarrollo y utilizando la rama "master" o "main" solo como puerta para hacer PUSH y PULL.
 - Haced PUSH de vuestras ramas de desarrollo. Ejemplo: push origin devManolo
 - **No debéis realizar un análisis funcional (AF).**
 - Entregad los **README.md**, de cada repositorio, correctamente rellenos, indicando los pasos necesarios para desplegar correctamente vuestro proyecto completo.
 - Entregad el script SQL de insert de Tarifas y/o cualquiera que necesite la aplicación para funcionar correctamente. Pensad que, lo que entreguéis, debe funcionar correctamente.
 - El documento PDF con las pruebas realizadas representadas mediante capturas de pantalla propiamente documentadas, así como los TOKEN (git clone), debe llamarse **PR_NombreGrupo_PI_1DAW.pdf**
- **El incumplimiento de alguna de estas normas será penalizado en la calificación.**

- **Rúbricas → Ver calificación en Aules**